

TP2 – File Integrity Checker and IOC Matcher in Rust

Student Name: Kadiata Niang

Student Matricule: 22017

Class: Master in Cybersecurity

Institution: Institut Supérieur du Numérique (SupNum), Mauritania

Academic Year: 2025–2026

Module: Module 7.1 – Programming with Rust

Instructor: S. Biha

GitHub Repository URL: https://github.com/BebeNiang/tp2_integrity_checker.git

Date: June 15, 2026

1. Objective

The primary objective of this laboratory assignment is to engineer a secure, resilient, and high-performance Command Line Interface (CLI) utility in Rust designed for automated endpoint defense and threat hunting workflows.

In defensive cybersecurity, File Integrity Checking is a fundamental control used to monitor critical system binaries and configuration files. It detects whether unauthorized modifications, backdoors, or malicious implants have been introduced by an adversary.

An Indicator of Compromise (IOC) is an artifact or technical evidence observed on a network or operating system that signifies a high probability of a security breach. In this utility, IOCs are represented as known malicious cryptographic SHA-256 signatures. By systematically hashing local file systems and cross-referencing them against an immutable list of trusted threat intelligence IOCs, security teams can rapidly isolate malicious droppers or altered components before execution.

2. Environment

The deployment and execution of this utility were strictly performed inside the standardized virtualized environment provided for the course:

Virtualization Platform: Docker Compose managing the isolated "rustlab" Debian-based container.

Project Directory Path: /workspace/tp2_integrity_checker inside the container.

Environment Control Commands:

```
mkdir -p workspace
```

```
docker compose up -d --build
```

```
docker compose exec rustlab bash
```

Toolchain State Evidence:

Rust Compiler: rustc 1.82.0

Package Manager: cargo 1.82.0

Static Analysis Engines: cargo clippy, cargo fmt

3. Implementation

The application is split into highly decoupled, modular components located under the `src/` tree:

Command-Line Parsing (`main.rs`): Orchestrates the entry point. It parses user-supplied arguments (`--target`, `--ioc`, and `--report`) dynamically through an iterative loop and natively intercepts the bonus parameters (`--json` and `--only-matches`).

Cryptographic Engine (`hashing.rs`): Leverages the external "sha2" dependency. To protect kernel memory from exhaustion when handling massive files, it reads files sequentially in small 4096-byte (4KB) chunks, streaming them incrementally into the SHA-256 digest machine.

Threat Intelligence Parser (`ioc.rs`): Uses buffered readers (`BufReader`) to parse the target `iocs.txt` file line-by-line. It uses string manipulation to successfully drop user comment markers (`#`), strip out arbitrary whitespaces, and catch invalid signatures cleanly.

System Walking & Matching (`scanner.rs`): Implements an iterative recursive directory traversal mechanism that walks through nested subdirectories, computes the hash of every encountered regular file, and runs a pattern match comparison against the loaded memory vector of malicious IOC entries.

Evidence Reporting (`report.rs`): Handles dual-format outputs. It formats the accumulated vector of results into a standard comma-separated format (CSV), or structurally maps entries onto a custom JSON array format depending on user switches.

4. Execution Evidence

```
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$ rustc --version && cargo --version && cargo fmt --version && cargo clippy --version
rustc 1.94.1 (e408947bf 2026-03-25)
cargo 1.94.1 (29ea6fb6a 2026-03-24)
rustfmt 1.8.0-stable (e408947bfd 2026-03-25)
clippy 0.1.94 (e408947bfd 2026-03-25)
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$
```

Figure 1: Toolchain and Environment Sanity Check. Verifies that the Rust environment, compiler, and linters are functioning correctly inside the container.

```
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$ sha256sum samples/files/*
70bbeaa0f2d408a45827aa9d5bd58209564a5bd7b61d6c069267c9e9e35f97cd samples/files/clean_readme.txt
1888673b4c962129e57b54b81fda2f967c52f87c28e9d7908e5fba0dfed097e3 samples/files/notes.txt
44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b samples/files/suspicious_dropper.txt
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$
```

Figure 2: Manual Integrity Evaluation. Verifies the baseline cryptographic signatures of the three sample files using the core Linux utility.

```
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$ cargo run -- --target samples/files --ioc samples/iocs.txt --report reports/scan_report.csv
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.34s
Running `target/debug/tp2_integrity_checker --target samples/files --ioc samples/iocs.txt --report reports/scan_report.csv`
TP2 File Integrity Checker (with Bonus Options)
Summary:
* Files scanned: 3
* IOC entries loaded: 2
* Invalid IOC lines: 1
* Matches found: 1
* Errors: 0

Matches:
[ALERT] samples/files/suspicious_dropper.txt
SHA-256: 44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b
IOC label: Demo suspicious test sample

Report written successfully to reports/scan_report.csv
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$
```

Figure 3: Application Run Sequence. Demonstrates running the compiled application against the test directory. It shows the successful detection of suspicious_dropper.txt, the output summary, and the creation of the CSV report.

```
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$ cargo test
  Finished `test` profile [unoptimized + debuginfo] target(s) in 0.29s
  Running unittests src/main.rs (target/debug/deps/tp2_integrity_checker-ef6558d79bb582ba)

running 5 tests
test hashing::tests::test_is_valid_sha256 ... ok
test ioc::tests::test_is_valid_sha256_indirect ... ok
test scanner::tests::test_scan_non_existent ... ok
test report::tests::test_csv_report_creation ... ok
test report::tests::test_json_report_creation ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s

kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$
```

Figure 4: Automated Unit Test Verification. Proof of the successful execution of our 5 custom unit testing suites passing at 100% green without panics, fully satisfying laboratory constraints.

```
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$ cargo clippy -- -D warnings
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.47s
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$
```

Figure 5: Static Analysis Linting Checks. Confirms that the application code satisfies strict compiler safety directives and contains zero warnings or style violations.

```
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$ cat reports/scan_report.json
[
  {
    "path": "samples/files/suspicious_dropper.txt",
    "sha256": "44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b",
    "status": "MATCH",
    "label": "Demo suspicious test sample"
  }
]
kadiata@df6bc4d49b4c:/workspace/tp2_integrity_checker$
```

Figure 6: Bonus Features Validation. Evidence of successful targeted alert filtering and clean JSON structural format mapping output.

```
rust-venv > workspace > tp2_integrity_checker >  Cargo.toml

1  [package]
2  name = "tp2_integrity_checker"
3  version = "0.1.0"
4  edition = "2024"
5
6  [dependencies]
7  sha2 = "0.10"
```

Figure 7: Cargo.toml

5. Discussion

Fault-Tolerant IOC Parsing

Per the functional parameters of the lab, the file `samples/iocs.txt` contained a corrupted row string: `"INVALID_HASH_LINE_SHOULD_BE_IGNORED"`. A naive application would typically crash or fail during parsing. Our implementation handles this by wrapping the length validation and hex checks in a defensive boolean function (`is_valid_sha256`). If a line fails this check, the loop safely increments an `invalid_count` accumulator and instantly invokes a `continue` statement to parse the next line without interrupting execution.

Missing or Locked Target Files

When navigating foreign file systems, the scanner may encounter invalid symlinks, unreadable system files, or deleted paths. To prevent uncontrolled program crashes (panics), the application encapsulates file input/output in Rust's monadic `Result<T, std::io::Error>` type. Any intercepted filesystem error is caught gracefully, serialized into a clean error message string, and embedded inside a `ScanStatus::Error(String)` variant. This enables the scanner loop to continue analyzing the remaining files uninterrupted.

Cryptographic Superiority of SHA-256 Over MD5/SHA-1

Legacy digest routines like MD5 (128-bit) and SHA-1 (160-bit) are broken and completely banned in professional cybersecurity frameworks due to their vulnerability to collision attacks. Attackers can mathematically forge a malicious binary that produces an identical hash signature to a trusted system file, bypassing standard integrity controls. SHA-256 (256-bit) offers a massive bit-space complexity that remains globally secure against both pre-image and collision generation attempts, ensuring absolute trust in file identity validation.

Advanced Bonus Extensions

By adding support for `--only-matches`, the utility leverages Rust's `"Vector::retain"` predicate mechanism to filter out background noise (clean files), isolating positive high-fidelity threat entries. The introduction of the `--json` flag enables seamless interoperability with modern Security Information and Event Management (SIEM) data collectors like Splunk or ElasticSearch, converting a basic laboratory script into an enterprise-ready security asset.

6. Conclusion

This assignment successfully demonstrates the engineering of a secure, production-grade security tool in Rust. By strictly utilizing Rust's key design paradigms—such as explicit pattern matching, strong

data encapsulation via enums, safe memory scoping (Ownership/Borrowing), and avoiding any `.unwrap()` shortcuts—we achieved complete functional reliability. The application fully complies with all academic and security guidelines established for this laboratory module.